

模型

大语言模型（LLMs）能够像人类一样理解和生成文本，可用于撰写内容、翻译语言、总结信息和回答问题，而无需针对每项任务进行专门训练。

除了文本生成之外，许多模型还支持：

- **工具调用** - 调用外部工具并将结果用于响应中。
- **结构化输出** - 模型的响应被限制为遵循定义的格式。
- **多模态** - 处理和返回非文本数据，如图像、音频和视频。
- **推理** - 模型执行多步推理以得出结论。

大语言模型 是智能体的推理引擎，决策大脑。它们驱动 **agent** 的决策过程，决定调用哪些工具、如何解释结果以及何时提供最终答案。

不同模型在不同任务上表现出色——有些更擅长遵循复杂指令，有些更擅长结构化推理，有些支持更大的上下文窗口以处理更多信息。

基本用法

模型可以通过两种方式使用：

1. **与 **agent** 一起使用** - 创建 **agent** 时传入模型名称或实例。
2. **独立使用** - 模型可**直接调用（在 **agent** 循环之外）**，用于文本生成、分类或提取等任务，而无需 **agent** 框架。

同一模型接口在两种上下文中均适用。

初始化模型

初始化模型的本质就是把模型名称, base-url, api-key, temperature, max-token 等参数**传入类返回一个类实例**，然后便可以使用**类方法**如 `invoke`, `stream` 等调用模型。模型会自动把传入的**消息封装发送到指定的节点并返回响应**。

以 `LangChain` 框架为例，使用独立模型的最简单方法是使用 `init_chat_model` 从选择的提供商初始化一个模型（OpenAI 示例）：

```
from langchain.chat_models import init_chat_model
os.environ["OPENAI_API_KEY"] = "sk-..."
model = init_chat_model("openai:gpt-4.1")
```

```
from langchain_openai import ChatOpenAI
os.environ["OPENAI_API_KEY"] = "sk-..."
model = ChatOpenAI(model="gpt-4.1")
```

参数

聊天模型接受可用于配置其行为的一组参数。支持的参数集因模型和提供商而异，但标准参数包括：

参数	类型	必填	说明
model	string	是	使用的特定模型的名称或标识符。
api_key	string	否	用于向模型提供商进行身份验证的密钥。通常通过设置环境变量访问。
temperature	number	否	控制模型输出的随机性。值越高，响应越具创造性；值越低，响应越确定性。
timeout	number	否	在取消请求之前等待模型响应的最大时间（秒）。
max_tokens	number	否	限制响应中的 token 总数，有效控制输出长度。
max_retries	number	否	如果因网络超时或速率限制等问题而失败，系统将重新发送请求的最大尝试次数。

每个聊天模型集成可能具有用于控制提供商特定功能的额外参数。例如，ChatOpenAI 具有 use_responses_api 以决定是否使用 OpenAI Responses 或 Completions API。

模型调用 **invoke&stream&batch**

必须调用聊天模型以生成输出。主要有三种调用方法，每种方法适用于不同的用例。

方法	说明
Invoke	模型接受消息作为输入，并在生成完整响应后输出消息。
Stream	调用模型，但实时流式传输生成的输出。
Batch	将多个请求批量发送给模型，以实现更高效的处理。

Invoke

调用模型的最直接方法是使用 invoke() 传入单个消息或消息列表。

```
response = model.invoke("为什么鹦鹉有五颜六色的羽毛？ ")
print(response)
```

可以向模型提供**消息列表**以表示对话历史。每条消息都有一个角色，模型用它来指示对话中谁发送了消息。

```
conversation = [
    {"role": "system", "content": "你是一个将英语翻译成法语的有用助手。"},
    {"role": "user", "content": "翻译：我喜欢编程。"},
    {"role": "assistant", "content": "J'adore la programmation."},
    {"role": "user", "content": "翻译：我喜欢构建应用程序。"}]
response = model.invoke(conversation)
print(response) # ("J'adore créer des applications.")
```

Stream ()

大多数模型可以在生成时流式传输其输出内容。通过逐步显示输出，流式传输显著改善了用户体验，尤其是对于较长的响应。

调用 `stream()` 返回一个迭代器，与返回单个 `AIMessage` 的 `invoke()` 不同，`stream()` 返回多个 `AIMessageChunk` 对象，每个对象包含一部分输出文本。可以使用循环实时处理每个块：

```
for chunk in model.stream("为什么鹦鹉有五颜六色的羽毛? "):
    print(chunk.text, end="|", flush=True)
```

重要的是，流中的每个块都设计为通过**求和聚合成完整消息**：

```
full = None # None | AIMessageChunk
for chunk in model.stream("天空是什么颜色? "):
    full = chunk if full is None else full + chunk
    print(full.text)# 天空# 天空是# 天空通常# 天空通常是蓝色# ...
    print(full.content_blocks)# [{"type": "text", "text": "天空通常是蓝色..."}]
```

生成的消息可以与使用 `invoke()` 生成的消息相同处理——例如，它可以聚合成消息历史并作为对话上下文传递回模型。

流式传输仅在程序的所有步骤都知道如何处理块流时才有效。例如，无法流式传输的应用程序是需要先将整个输出存储在内存中才能处理的情况。

Batch ()

将一组独立请求批量处理给模型可以显著提高性能并降低成本，因为处理可以并行进行：

```
responses = model.batch([
    "为什么鹦鹉有五颜六色的羽毛?",
    "飞机是如何飞行的?",
    "什么是量子计算?"])
for response in responses:
```

```
print(response)
```

本节描述的是对话模型的方法 `batch()`，它在客户端**并行化模型调用**。它不同于推理提供商支持的批量 API。

默认情况下，`batch()` 仅返回**整个批次的最终输出**。如果希望在每个单独输入完成生成时接收输出，可以使用 `batch_as_completed()` **流式传输**结果：

```
for response in model.batch_as_completed([ "为什么鹦鹉有五颜六色的羽毛？ ",
"飞机是如何飞行的？ ", "什么是量子计算？ "]):
    print(response)
```

使用 `batch_as_completed()` 时，结果可能**无序到达**。每个结果包括输入索引，以便根据需要匹配和重建原始顺序。

在使用 `batch()` 或 `batch_as_completed()` 处理大量输入时，若希望控制最大并行调用数。可以通过在 **RunnableConfig** 字典中设置 `max_concurrency` 属性来完成。

```
model.batch( list_of_inputs, config={ 'max_concurrency': 5, # 限制为 5 个并行调用 })
```

消息

消息是模型上下文的基本单位。它们**代表模型的输入和输出**，携带内容和元数据，用于在与 LLM 交互时表示对话状态。相比于纯文本，消息一般是包含角色和内容等字段的字典格式，注意字段的名称和结构不同的模型可能会有区别。

消息是包含以下内容的对象：

- **角色** - 标识消息类型（例如 `system`、`user`）
- **内容** - 表示消息的实际内容（例如文本、图像、音频、文档等）
- **元数据** - 可选字段，例如响应信息、消息 ID 和令牌使用情况

```
conversation = [
{"role": "system", "content": "你是一个将英语翻译成法语的有用助手。"},
{"role": "user", "content": "翻译：我喜欢编程。"},
{"role": "assistant", "content": "J'adore la programmation."},
{"role": "user", "content": "翻译：我喜欢构建应用程序。"}]
```

LangChain 提供了一种**标准消息类型**，可在所有模型提供商之间工作，确保无论调用哪个模型都能保持一致的行为。

消息类型

1.系统提示消息 SystemMessage:

`SystemMessage` 表示一组初始指令，设置语气、定义模型角色并建立响应指南，用于引导模型的行为并为交互提供上下文。

```
system_msg = SystemMessage("You are a helpful coding assistant.")
```

`SystemMessage` 类强制规定了 `role` 元数据为 `"system"`。

内部数据结构：当你把这行代码发送给模型 API 时，它会被序列化为：

```
{ "role": "system", "content": "You are a helpful coding assistant." }
```

2. 人类消息 `HumanMessage`

`HumanMessage` 表示用户输入和交互。它们可以包含文本、图像、音频、文件以及任何其他多模态内容。

2.1 文本内容

方式 A：显式声明角色

```
response = model.invoke([ HumanMessage("What is machine learning?") ])
```

方式 B：字符串快捷方式

```
response = model.invoke("What is machine learning?")
```

方式 B 是语法糖：框架（如 `LangChain`）在底层检测到传入的不是消息列表，而是一个孤立的字符串时，会执行**隐式自动包装**：

底层近似逻辑

```
if isinstance(input, str):
    input = [HumanMessage(content=input)]
```

· **关键差异与风险：**

方式 A：你是上下文状态的构建者。你知道这里只有一条人类消息，没有历史。

方式 B：你是**单次问答**的调用者。虽然方便，但如果后续你想传入 `SystemMessage` 或历史对话，**必须切换回方式 A** 传入完整列表。它剥夺了你管理上下文的权限。

2.2 消息元数据

```
human_msg = HumanMessage(
```

```
content="Hello!",  
name="alice", # 可选: 标识不同用户  
id="msg_123", # 可选: 用于追踪的唯一标识符)
```

这两个字段在复杂的多智能体或审计追踪系统中至关重要。

name 字段: 区分同角色下的不同实体

虽然角色是 human, 但对话里可能不止一个人类。

场景: 客服转接记录

如果系统需要记录客户和转接后的人工客服都在输入:

```
messages = [  
HumanMessage(content="我的快递丢了", name="customer_zhang"), # 角色  
human, 实体张三  
AIMessage(content="请提供单号", name="bot_001"),  
HumanMessage(content="单号是 SF123", name="agent_li") # 角色 human, 实体李  
客服]
```

模型感知: 虽然 OpenAI 等 API 目前不直接根据 name 改变回复逻辑, 但高级模型可以利用这个元数据区分说话者身份, 从而生成更准确的指代 (例如: “正如李客服刚才提到的...”)。

id 字段: 消息的“身份证”与审计链条

作用 1: 去重与幂等性

当网络波动导致重试时, 通过检查 id 可以避免同一条消息被重复计入上下文窗口, 防止浪费 Token。

作用 2: 反馈链路追踪

在企业级应用中, 如果你需要根据模型输出来优化上游指令, 你需要知道模型是基于哪几条特定历史消息的 ID 做出了这个判断。这在 LangSmith 或 LangFuse 等观测平台中是标准实践。

注意

name 字段的行为因提供商而异 - 有些用于用户识别, 其他忽略它。

3.AI 消息 AIMessage

AIMessage 表示模型调用的输出 response。它们可以包含文本内容, 多模态数据、工具调用指令和提供商特定的元数据。

```
response = model.invoke("Explain AI")
print(type(response)) # <class 'langchain_core.messages.AIMessage'>
```

`AIMessage` 对象由调用模型时返回，其中包含响应中的所有关联元数据。

可以手动创建新的 `AIMessage` 对象并将其插入消息历史中。

```
# 手动创建 AI 消息（例如，用于对话历史）
ai_msg = AIMessage("I'd be happy to help you with that question!")
# 添加到对话历史
messages = [
    SystemMessage("You are a helpful assistant"),
    HumanMessage("Can you help me?"),
    ai_msg, # 插入就像来自模型一样
    HumanMessage("Great! What's 2+2?")]
response = model.invoke(messages)
```

属性

- **text** (string): 消息的文本内容。
- **content** (string | dict[]): 消息的原始内容。
- **content_blocks** (ContentBlock[]): 消息的标准化内容块。
- **tool_calls** (dict[] | None): 模型进行的工具调用。如果没有调用工具，则为空。

```
model_with_tools = model.bind_tools([get_weather])
response = model_with_tools.invoke("What's the weather in Paris?")
for tool_call in response.tool_calls:
    print(f'Tool: {tool_call["name"]}')
    print(f'Args: {tool_call["args"]}')
    print(f'ID: {tool_call["id"]}')
```

- **id** (string): 消息的唯一标识符（由 LangChain 框架等自动生成或在提供商响应中返回）

- **usage_metadata** (dict | None): 消息的使用元数据，可包含可用时的令牌计数。

```
response = model.invoke("Hello!")
response.usage_metadata
{'input_tokens': 8, 'output_tokens': 304, 'total_tokens': 312, 'input_token_details':
{'audio': 0, 'cache_read': 0}, 'output_token_details': {'audio': 0, 'reasoning': 256}}
```

- **response_metadata** (ResponseMetadata | None): 消息的响应元数据。

4. 工具消息 ToolMessage

工具消息用于将单个工具执行的结果传回模型。

属性

content (string, 必需): 工具调用的字符串化输出。

tool_call_id (string, 必需): 此消息响应的工具调用的 ID。(必须匹配 AIMessage 中的工具调用 ID)

name (string, 必需): 被调用的工具的名称。

artifact (dict): 不发送给模型但可以以编程方式访问的附加数据。这对于存储原始结果、调试信息或下游处理的数据而不会使模型上下文杂乱很有用。

工具 可以直接生成 ToolMessage 对象。

```
# 执行工具并创建结果消息
weather_result = "Sunny, 72° F"
tool_message = ToolMessage(content=weather_result,
                             tool_call_id="call_123" # 必须匹配调用 ID)
# 继续对话
messages = [
    HumanMessage("What's the weather in San Francisco?"),
    ai_message, # 模型的工具调用
    tool_message, # 工具执行结果]
response = model.invoke(messages) # 模型处理结果
```

基本用法

1. 文本提示

文本提示是字符串 - 适用于不需要保留对话历史的简单生成任务。

```
response = model.invoke("Write a haiku about spring")
```

何时使用文本提示:

- 只有一个独立的请求
- 不需要对话历史
- 希望代码复杂度最小消息提示

2. 消息提示

可以通过创建消息对象，并在调用时将消息对象列表传递给模型。

```
from langchain.messages import SystemMessage, HumanMessage, AIMessage
```



```
messages = [
    SystemMessage("You are a poetry expert"),
    HumanMessage("Write a haiku about spring"),
    AIMessage("Cherry blossoms bloom...")]
response = model.invoke(messages)
```

何时使用消息提示：

- 管理多轮对话
- 处理多模态内容（图像、音频、文件）
- 包含系统指令

3.字典格式

还可以直接以 OpenAI 聊天补全格式指定消息。

```
messages = [
    {"role": "system", "content": "You are a poetry expert"},
    {"role": "user", "content": "Write a haiku about spring"},
    {"role": "assistant", "content": "Cherry blossoms bloom..."}]
response = model.invoke(messages)
```

上下文窗口

Agent 的上下文窗口并非一个简单的“容器”，它更像是一个动态调配的“工作台”，里面汇聚了让模型精准执行任务的各类关键信息。如果将一个强大的 Agent 比作“CPU”，上下文窗口就是它的“RAM”（工作内存），是进行实时计算的核心区域。

你看到的上下文窗口无论多么复杂，最终在模型眼里就是一串顺序严格的 messages 数组。所有复杂的内容类型（RAG、记忆、工具、技能），本质上都被包装成了这 4 种标准角色的 content 字段。

角色 (role)	承载的内容类型	精简功能介绍
system	1. 系统提示词 (System Prompt)	定义 Agent 的基础人设、语气、全局行为准则和核心禁忌。这是模型最高优先级的元指令。
system	2. 工具定义列表 (Tools Schema)	以 JSON Schema 或纯文本形式描述可用工具的名称、用途及参数。让模型知道“我能调用什么外部功能”。

角色 (role)	承载的内容类型	精简功能介绍
system	3. 用户偏好	关于当前用户的持久化偏好、背景信息（如“用户名 叫张三，喜欢 Python”）。用于实现跨会话的个性化 体验。
system	4. Skill 元数据列 表	所有可用技能包的“名片夹”（仅包含技能名和一句话 描述）。用于引导模型在任务规划初期发现并激活正 确的能力包。
user	5. 激活的 Skill 详细指令	当模型决定使用某个技能后，该技能的完整操作手册 （SKILL.md 正文）被一次性拼接至此。提供执行复 杂任务的标准流程。
user	1. 用户原始输入	驱动本次对话回合的直接指令或提问。
user	2. RAG 检索增强 内容	从外部向量数据库召回的相关知识片段。通常以“请 参考以下资料回答”的前缀格式拼接在用户问题之 前，用于对抗幻觉、注入实时信息。
user	3. 文件上传内容	用户上传的 PDF、图片、代码文件的解析文本。作 为原始输入的附件上下文一起提交。
assistant	1. 模型文本回复	模型针对用户问题生成的、展示给用户的最终答案。
assistant	2. 模型思考过程 (CoT)	在推理模型中，用于记录 think 块内的逻辑推导链条 （例如 DeepSeek-R1 或 OpenAI o1 的思考过程）。
assistant	3. 工具调用请求 (Tool Call)	特殊的非文本内容。Assistant 消息中包含 tool_calls 字段，列出模型想要执行的函数名和参数。这是 Agent 采取行动的“中枢神经信号”。
assistant	4. 伪造的历史对 话 (Few-Shot)	作为少样本示例被硬编码进消息列表的历史问答对。 用于约束模型的输出格式或行为模式。
tool	1. 工具执行结果 (Function Output)	外部函数（如搜索引擎、计算器）返回的具体数据或 报错信息。模型收到此角色消息后，会基于结果继续 生成下一轮的自然语言回复或进行下一次调用。
tool	2. Skill 脚本执行 反馈	如果 Skill 内部包含可执行脚本（如 Python 数据处 理脚本），其运行后的标准输出会作为 tool 消息返 回，供模型整合进最终答案。

工具

Agent 工具的本质函数或类方法，实现具体业务逻辑（如网页检索、数据库
查询等），由 agent 统一封装为 tools。

精准的工具描述和规范的参数定义是交互顺畅的关键。

工具的关键要素

名称：唯一标识，Agent 通过名称引用工具。

描述：Agent 的 LLM 根据描述判断何时使用该工具。描述应清晰说明工具的功能、适用场景及输入格式。

输入参数：通常使用 Pydantic 模型定义，确保类型安全和自动校验。

执行逻辑：即工具的具体功能，可以访问外部 API、操作数据库、运行本地代码等。

Skills

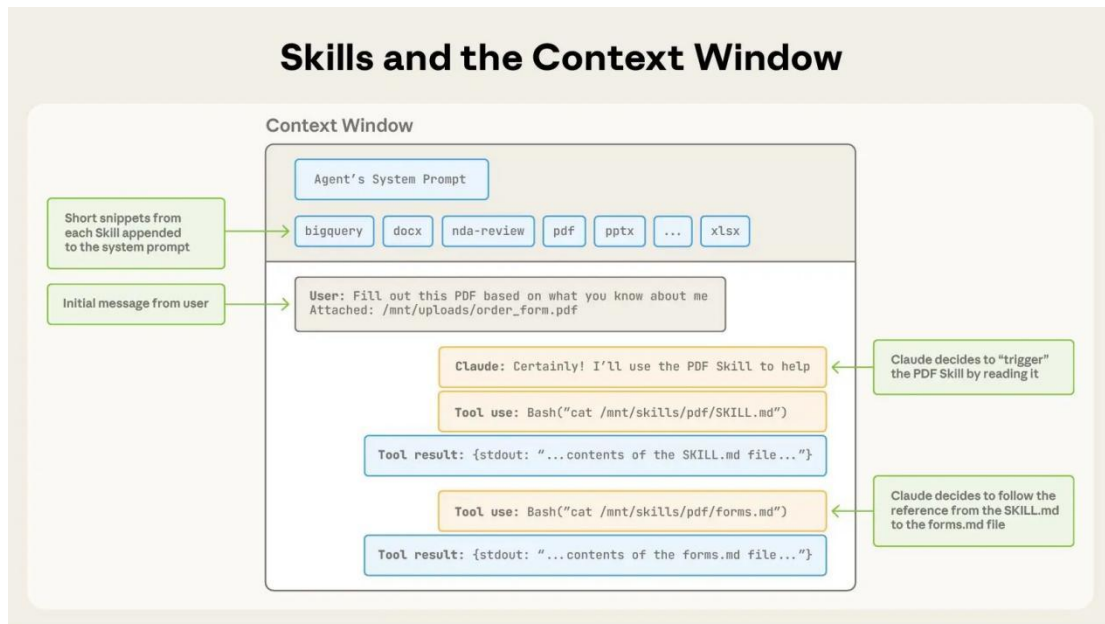
Skills 是把做某件事的过程概括为**标准执行步骤（SOP）**，把该步骤注入到大模型的当前 prompt 里，便可以指导大模型完成任务，省去了大模型自己规划的不确定性。任务完成后即使用新的 prompt，减少过长的 prompt 带来的指令漂移等现象。

这就是 Anthropic 提出 Skills 的核心思想：

与其堆叠更多“做什么”的工具，不如把“怎么做”的经验，打包成可复用的“知识胶囊”，让 Agent 在需要时加载并遵循它。

每一个 skill 其实就是一个文件夹，标准文件结构如下：

```
my-skill/
├── SKILL.md           # 核心提示词与指令
├── scripts/           # 可执行脚本 (Python/Bash)
├── references/        # 需加载到上下文的参考文档
└── assets/           # 静态资源与模板
```



为了更好地节省 prompt 的 token 数, skill 设计了渐进式披露机制, 逐步加载。就像一本组织良好的手册, 从目录开始, 然后是具体章节, 最后是详细的附录一样, 技能允许 agent 仅在需要时加载信息。

记忆

基本类型和管理方式

一个成熟的 Agent 往往至少有 2 层记忆:

1. 会话短期记忆: messages
 - 作用: 保留对话上下文、工具结果、用户指令。
 - 特点: 最贴近模型, 但容易膨胀。
2. 持久化/长期记忆: 例如 checkpointer、sysprompt memory block、专家经验库
 - 作用: 跨轮次、跨会话复用历史经验。
 - 特点: 必须治理, 否则会污染后续任务。

记忆管理不只是存储: 五个管理维度

写入 (Write) 何时写入记忆, 决定记忆是否有价值。例如:

- 人工修复完成后，把 `pending\_expert\_save` 写入专家经验库。

检索（Retrieve）检索决定记忆是否真正被使用。例如：

- 根据 error pattern 检索专家建议。

压缩（Compress）长会话 messages 会越来越长，必须压缩，否则成本和噪声都会上升。

- 清理一部分原始消息，只保留关键事件。

淘汰（Evict）记忆不是越多越好。没有淘汰机制，系统很快会被过期、错误记忆污染。可淘汰对象包括：

- 低置信度专家经验；

隔离（Isolation）记忆必须按作用域隔离，否则会产生严重串扰。

用 `user\_id` 与 `session\_id` 组合状态数据库路径。

记忆管理示例代码片段

```
class MemoryManager:
    def __init__(self, max_tokens=4000):
        self.short_term = []
        self.long_term_store = {} # 实际应使用向量数据库

    def add_interaction(self, user_msg, assistant_msg):
        self.short_term.append({"user": user_msg, "assistant": assistant_msg})
        # 如果超长则进行压缩或遗忘
        if self._estimate_tokens() > self.max_tokens:
            self._compress()

    def _compress(self):
        # 调用 LLM 生成摘要
        summary = "用户询问了天气，助手回答了晴朗。"
        self.short_term = [{"summary": summary}]
```